Disegnare con i cicli: i for annidati

Lezione 13 — 1IOT, SST-ICT-01 — Esercitazione: asterischi, rettangolo pieno e vuoto, triangolo

20/10/2026

1 Che cosa abbiamo fatto oggi

Oggi abbiamo usato i costrutti della Lezione 12 (**for**, **while**, **do while**) in una **serie di esercizi**, ognuno un po' più difficile del precedente: non ce n'è mai uno **più facile** di quello prima. Si parte da una riga di asterischi, si arriva a un **rettangolo**, poi a un rettangolo **vuoto** e infine a un **triangolo**. In tutti gli esercizi c'è anche il problema di **controllare il dato** che inserisce l'utente.

- Ogni esercizio si scrive in un **nuovo progetto** su onlinegdb: quello già fatto **non si modifica**, si tiene.
- Prima di andare avanti si **prova** quello che si è scritto, anche se il programma non è ancora finito: così gli errori si trovano subito, quando il codice è ancora corto.
- Il codice va sempre **indentato** (Sezione 4.4).

Nota

Il carattere che si stampa non ha importanza: l'**asterisco** si usa per tradizione, ma va benissimo una `x`, una chiocciola `@` o qualunque altro carattere.

2 Esercizio 1: una riga di asterischi

Esempio

Testo. Scrivere un programma che chieda all'utente di inserire un numero **maggiore di 0**. Il programma poi stampa **tanti asterischi** quanti specificati nel numero.

2.1 Il dato va controllato

Per far entrare un dato nella macchina **durante l'esecuzione** del programma (si dice *a runtime*) servono, come sempre, un `printf` che dice all'utente che cosa vogliamo e uno `scanf` che **acquisisce** il dato.

La novità è che il numero deve essere **maggiore di 0**: sono ammessi 1, 2, 3, 4…, **non** sono ammessi 0, -1, -5. Se l'utente scrive un dato che non rispetta la richiesta, la macchina **non lo accetta** e **rifà la domanda**.

Qual è il costrutto più comodo? Il **do while**, perché la domanda va fatta **almeno una volta**. La condizione del `while` dice **quando ripetere**, cioè quando il dato è **sbagliato**: si ripete finché `N < 1`.

Attenzione

Perché `N < 1` e non `N < 0`? Con `N < 0` lo **zero** verrebbe accettato (0 non è minore di 0), ma noi lo zero **non lo vogliamo**. Le scritture giuste sono due, **equivalenti**: `N < 1` oppure `N <= 0`. Se ne sceglie una delle due.

Il `while` finale può stare sulla stessa riga della graffa chiusa oppure a capo: al C **non interessa** come vai a capo o come indenti, il programma funziona lo stesso. Potresti scrivere tutto su una riga sola, ma poi **non si legge**. Dentro le graffe del `do` ricordati sempre lo **spazio dell'indentazione**.

2.2 Proviamo subito la prima parte

Prima di scrivere il ciclo degli asterischi conviene mandare in esecuzione (**Run**) quello che c'è, per vedere se **non ci sono errori**. Il programma chiede il numero; chiaramente non stampa ancora gli asterischi, ma possiamo provare la domanda:

```
Inserisci il numero di asterischi da stampare: 0
Inserisci il numero di asterischi da stampare: -5
Inserisci il numero di asterischi da stampare: 5
```

Con 0 e con -5 la domanda si **ripete**: il dato non viene accettato. Con 5 il programma **esce dal ciclo** e arriva alla fine. Fin qui è corretto.

2.3 Gli asterischi: il for

Ora bisogna stampare `N` asterischi. Il numero delle ripetizioni lo **sappiamo già** (è `N`), quindi si usa il **for**. Serve una **nuova variabile** per contare: **non può essere** `N`, perché `N` la stiamo già usando come **limite** (fin dove arrivare). La chiamiamo `x`: parte da 0, si ripete finché `x < N`, e a ogni giro `x++`.

La variabile del ciclo si può dichiarare **direttamente dentro il** `for`, come facciamo in aula: `for (int x = 0; x < N; x++)`. Così `x` esiste **solo dentro il ciclo**, e non serve dichiararla in cima al programma.

Scritto così, però, il `for` **non fa ancora niente**: gli abbiamo detto quante volte ripetere, ma non **che cosa** ripetere. Dentro va la stampa di un asterisco, `printf("*");`.

```c
#include <stdio.h>

int main()
{
    int N;

    // chiedo il numero finche' non e' maggiore di 0
    do
    {
        printf("Inserisci il numero di asterischi da stampare: ");
        scanf("%d", &N);
    } while (N < 1);

    // stampo N asterischi sulla stessa riga
    for (int x = 0; x < N; x++)
    {
        printf("*");
    }
    // a capo finale (aggiunto in dispensa: il cursore non resta sulla riga)
    printf("\n");

    return 0;
}
```

```
Inserisci il numero di asterischi da stampare: 5
*****
```

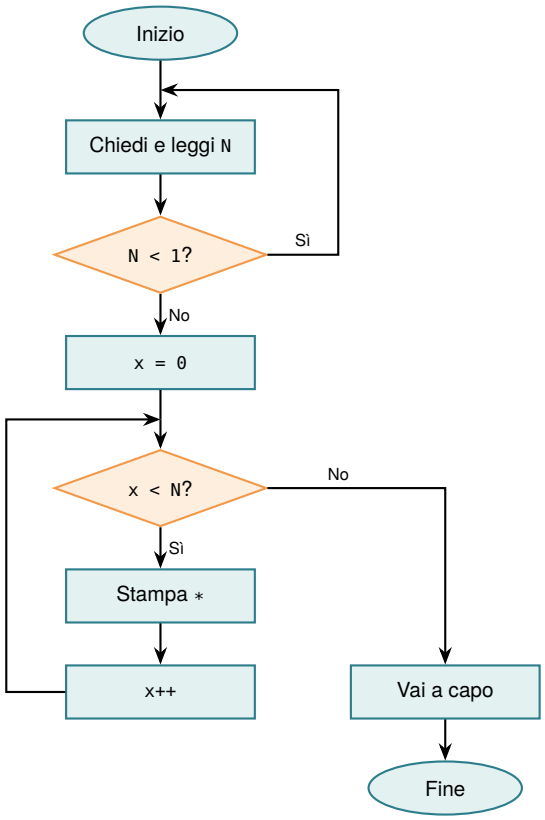


Figura 1: Esercizio 1: il `do while` del controllo del dato, poi il `for` degli asterischi.

2.4 Ripasso: do while e while

Il **do while** è una variante del `while` che ti assicura che il codice tra le graffe venga eseguito **almeno una volta**. Il `while`, invece, controlla la condizione **prima**: se è falsa al primo colpo, il codice non viene eseguito **nemmeno una volta**, e quindi la domanda all'utente non verrebbe mai fatta. Per questo per chiedere un dato si usa il `do while`.

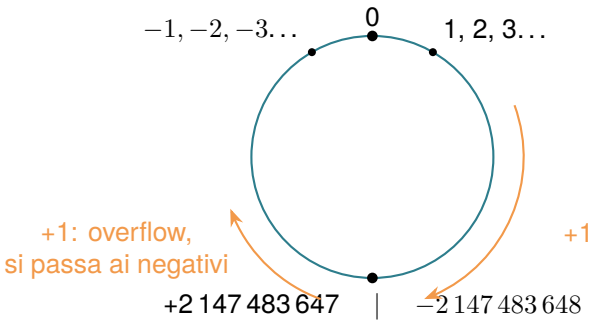
2.5 Una domanda dalla classe: e se il numero è troppo grande?

Provando il programma con numeri **molto grandi** succedono cose strane: a volte il programma rifà la domanda come se il numero fosse negativo, a volte comincia a stampare asterischi **senza fermarsi più**. Il motivo è il **tipo di dato**.

Un `int` ha una **dimensione in bit**: il C garantisce **almeno 16 bit**, ma la dimensione vera dipende dalla macchina. Sui PC di oggi (e su onlinegdb) un `int` è di **32 bit**. Con 32 bit le combinazioni sono 2^{32}, circa **4 miliardi** (4 294 967 296). Siccome l'`int` contiene numeri **positivi e negativi**, questo intervallo si divide a metà: si va da circa -2 miliardi a circa $+2$ miliardi (per la precisione da $-2\,147\,483\,648$ a $+2\,147\,483\,647$).

Definizione

I numeri interi nel computer sono organizzati come su un **cerchio**: si parte da 0, si sale 1, 2, 3... e a metà cerchio si arriva al massimo, circa **+2 miliardi**. Dall'altra parte ci sono i negativi, -1, -2, -3... fino a circa -2 **miliardi**. Se al numero più grande aggiungo 1, **non** ottengo un numero più grande: finisco a -2 miliardi. Dal punto di vista matematico è sbagliato: è l'**overflow** (traboccamento).



Una volta l'overflow era un problema contro cui si **sbatteva continuamente**; con le macchine di oggi è più difficile arrivarci, ma **non è impossibile**. Nel nostro programma:

- con **3 miliardi** l'`int` contiene un numero **negativo** ($-1\,294\,967\,296$): il `do while` lo considera sbagliato e **rifà la domanda**;
- con **5 miliardi** l'`int` contiene un numero **positivo** (705 032 704): il programma lo accetta e stampa **oltre 700 milioni** di asterischi, e sembra che non si fermi più.

Nota

Approfondimento. In aula abbiamo visto che con un numero oltre i 2 miliardi il programma rifà la domanda, come se fosse negativo. I due casi qui sopra (3 e 5 miliardi) sono stati provati sul PC dopo la lezione, per spiegare anche il caso del programma che "non si ferma più". Nel C standard, superare il limite di un `int` con segno è un *comportamento non definito*: il "giro sul cerchio" è quello che fanno in pratica i nostri computer. Il valore esatto non serve ricordarlo: l'importante è sapere che ogni **tipo di dato** ha un **limite**.

2.6 E se scrivo una lettera?

Nello `scanf` abbiamo scritto `%d`: gli abbiamo detto di acquisire un **numero intero**. Se l'utente scrive una **lettera** (o il proprio nome), lo `scanf` **non riesce** a leggerla, la variabile non riceve un valore valido e la lettera resta lì, non letta: il programma può rifare la domanda **all'infinito** o comportarsi in modo strano. Per ora sappiamo acquisire **solo numeri**; per acquisire **parole** servono strumenti che vedremo più avanti, indicativamente verso **novembre**.

3 Esercizio 2: il rettangolo di asterischi

Esempio

Testo. Scrivere un programma che chieda all'utente di inserire **due numeri maggiori di 0**, `C` e `R`. Il programma stampa un **rettangolo** fatto di asterischi, **largo** `C` e **alto** `R`.
Esempio: se `C` vale 5 e `R` vale 3, il programma stampa

```
*****
*****
*****
```

I nomi aiutano a ricordare: `C` sta per **colonne** (la larghezza), `R` per **righe** (l'altezza).

3.1 Le due domande

Prima di tutto si dichiarano le due variabili, `int C` e `int R`. Poi le due domande, ognuna con il suo **do while** (si ripete finché il numero è minore di 1), proprio come nell'Esercizio 1.

Attenzione

Contestualizza la richiesta. Il primo numero rappresenta la **larghezza** del rettangolo: allora il `printf` non dice "Inserisci il primo numero", ma `"Inserisci la larghezza del rettangolo"`. Il secondo: `"Inserisci l'altezza del rettangolo"`. L'utente deve capire **che cosa** gli stai chiedendo.

Il `for` non si mette mentre si stanno ancora chiedendo i dati: in quel momento non si sa ancora **quante volte** ripetere. Si scrive **dopo** che tutti e due i dati sono stati acquisiti.

3.2 Dividere il problema in due

Stampare un rettangolo è un po' più complesso dell'Esercizio 1. Conviene **dividere il problema** in due parti.

1. **Prima parte:** stampare **una riga** di `C` asterischi. Questo lo sappiamo già fare: è l'Esercizio 1, un `for` con `x` che va da 0 finché `x < C`, e dentro `printf("*");`.
2. **Seconda parte:** questa riga va stampata **`R` volte** (nell'esempio, 3). Potrei scrivere tre volte lo stesso `for` con il copia e incolla, ma se `R` vale 50? **Non si fa così**: tutto il blocco del `for` si mette **dentro un altro `for`**, che lo ripete `R` volte. Serve una **variabile diversa**: `y`, che va da 0 finché `y < R`.

Così il programma ripete **3 volte** una cosa che ripete **5 volte**. Ti ricordi come si chiama un `if` dentro un altro `if`? **Annidato**. Allo stesso modo, un `for` dentro un altro `for` si chiama **for annidato** (o ciclo annidato). Il `for` con `y` è il ciclo **esterno** (le righe), quello con `x` è il ciclo **interno** (le colonne).

3.3 Il primo tentativo: manca qualcosa

Scrivendo solo i due `for`, uno dentro l'altro, con il `printf("*");` nel più interno, il codice è **scritto giusto** (non ci sono errori), ma **non fa ancora** quello che vogliamo: con 5 e 3 non esce un rettangolo, ma **un'unica riga** di 15 asterischi.

****

Che cosa manca? Ogni volta che finiamo di scrivere **una riga** dobbiamo **andare a capo**. Non a ogni asterisco, ma **ogni 5 asterischi**, cioè quando il ciclo interno è finito. Quindi il `printf("\n");` va **dopo il for interno**, ma ancora **dentro il for esterno**.

```c
#include <stdio.h>

int main()
{
    int C;
    int R;

    // larghezza del rettangolo (colonne)
    do
    {
        printf("Inserisci la larghezza del rettangolo: ");
        scanf("%d", &C);
    } while (C < 1);

    // altezza del rettangolo (righe)
    do
    {
        printf("Inserisci l'altezza del rettangolo: ");
        scanf("%d", &R);
    } while (R < 1);

    // ciclo esterno: una volta per ogni riga
    for (int y = 0; y < R; y++)
    {
        // ciclo interno: una riga di C asterischi
        for (int x = 0; x < C; x++)
        {
            printf("*");
        }
        // finita la riga, a capo
        printf("\n");
    }

    return 0;
}
```

```
Inserisci la larghezza del rettangolo: 5
Inserisci l'altezza del rettangolo: 3
*****
*****
*****
```

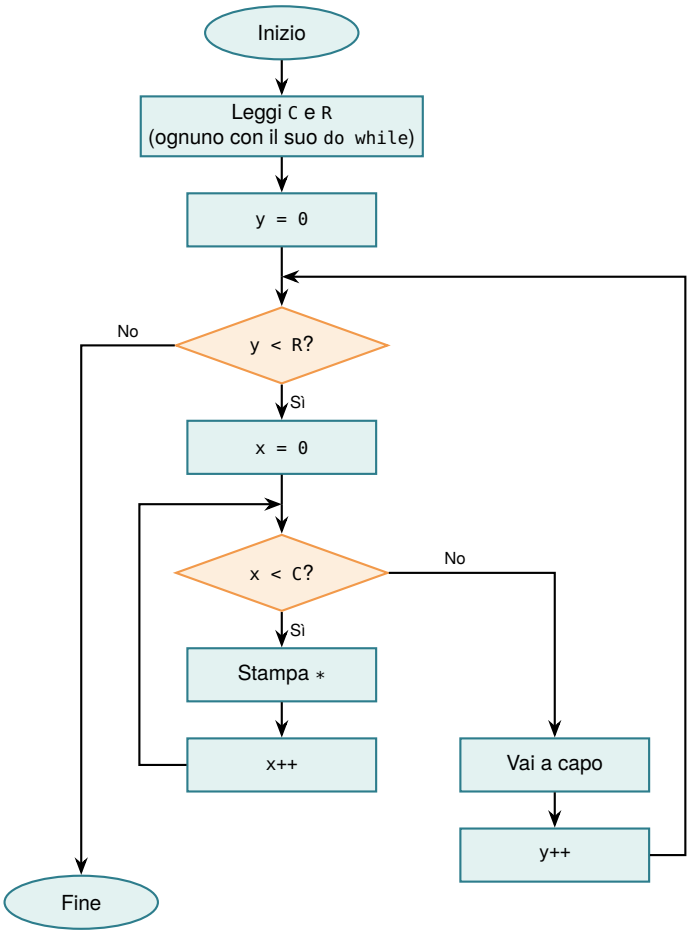


Figura 2: Esercizio 2: due cicli annidati. Il ciclo interno (x) stampa una riga; il ciclo esterno (y) la ripete R volte, andando a capo dopo ogni riga.

4 Esercizio 3: il rettangolo vuoto

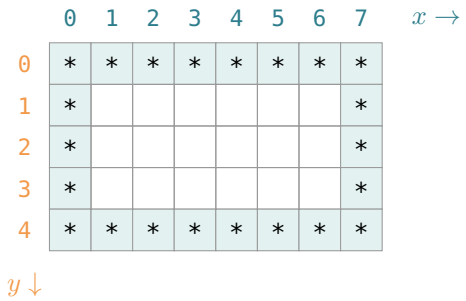
Finito il rettangolo pieno, dalla classe è arrivata una richiesta: **e se lo volessi vuoto**, cioè con gli asterischi **solo sul bordo**? È la strada giusta: ogni giorno faremo codici **sempre più complicati**. Proviamo.

Esempio

Testo. Modificare il programma dell'Esercizio 2 (in aula si è lavorato sullo stesso programma) in modo che il rettangolo sia **vuoto**: gli asterischi si stampano **solo sul bordo**, dentro ci sono degli **spazi**.

4.1 Le coordinate, come a battaglia navale

Disegniamo un rettangolo più grande, **8 per 5** (`C` = 8, `R` = 5), e diamo un numero a ogni colonna e a ogni riga, come nella **battaglia navale**. Le colonne sono la `x`, che va da 0 a 7 verso destra; le righe sono la `y`, che va da 0 a 4 verso il basso. Si parte da **0** perché i nostri `for` partono da 0.



Quando si stampa l'asterisco?

- quando `y` vale **0**: tutta la **prima riga** si stampa;
- quando `x` vale **0**: tutta la **prima colonna** si stampa;
- nell'**ultima riga**. Quanto vale `y` nell'ultima riga? Nel disegno 4, ma **non sempre** 4: se l'altezza è 5 l'ultima riga è la 4, se l'altezza è 10 è la 9, se è 18 è la 17. In generale l'ultima riga è **`R - 1`**, perché si parte a contare da 0;
- nell'**ultima colonna**, per lo stesso motivo: **`x == C - 1`**.

Il `printf` dell'asterisco, nel ciclo interno, va quindi sotto un `if`: stampa se `x == 0` **oppure** `y == 0` **oppure** `y == R - 1` **oppure** `x == C - 1`. L'"oppure" è l'operatore **OR**, `||`.

4.2 Una condizione alla volta

Con le prime due condizioni (`x == 0 || y == 0`) il programma stampa davvero la prima riga e la prima colonna. Aggiungiamo l'ultima riga, `y == (R - 1)`, e proviamo con 8 e 5:

```
********
*
```

```
*
*
********
```

C'è anche la riga sotto. Che cosa manca? L'**ultima colonna**: `x == (C - 1)`.

4.3 Il tentativo senza else: gli asterischi si ammucchiano

Aggiungendo anche l'ultima colonna, però, il risultato **non è quello che vogliamo**: la logica è ineccepibile, la macchina ha fatto **esattamente quello che abbiamo scritto**, ma l'ultima colonna **non si trova** al suo posto: gli asterischi finiscono **attaccati** a quelli della prima colonna.

```
********
**
**
**
********
```

Il problema è che ci siamo dimenticati di dire che cosa fare quando **tutte le condizioni sono false**. Se non gli diciamo niente, il programma **non stampa niente**: stampa l'asterisco della prima colonna e, quando `x` arriva a 7, stampa il secondo **subito di fianco**. Invece, ogni volta che le condizioni non sono verificate, deve stampare uno **spazio**: lo spazio non si vede, ma **occupa il posto** e permette all'asterisco dell'ultima colonna di arrivare dove vogliamo. Basta aggiungere un **else** che stampa uno spazio.

```
    for (int y = 0; y < R; y++)
    {
        for (int x = 0; x < C; x++)
        {
            // asterisco solo sui bordi, altrimenti uno spazio
            if (x == 0 || y == 0 || y == (R - 1) || x == (C - 1))
            {
                printf("*");
            }
            else
            {
                printf(" ");
            }
        }
        printf("\n");
    }
```

Le parentesi intorno a `R - 1` e `C - 1` non sono obbligatorie (il confronto `==` viene fatto dopo la sottrazione), ma rendono la condizione **più leggibile**: in aula le abbiamo messe.

```
Inserisci la larghezza del rettangolo: 8
Inserisci l'altezza del rettangolo: 5
********
*      *
*      *
*      *
********
```

Nota

Per vedere gli spazi. Prova a stampare un **puntino** al posto dello spazio, `printf(".");`: il rettangolo viene stampato con i puntini in mezzo. Quei puntini sono i **posti vuoti** che non vogliamo riempire: per questo, nella versione finale, al loro posto c'è uno spazio.

```
********
*......*
*......*
*......*
********
```

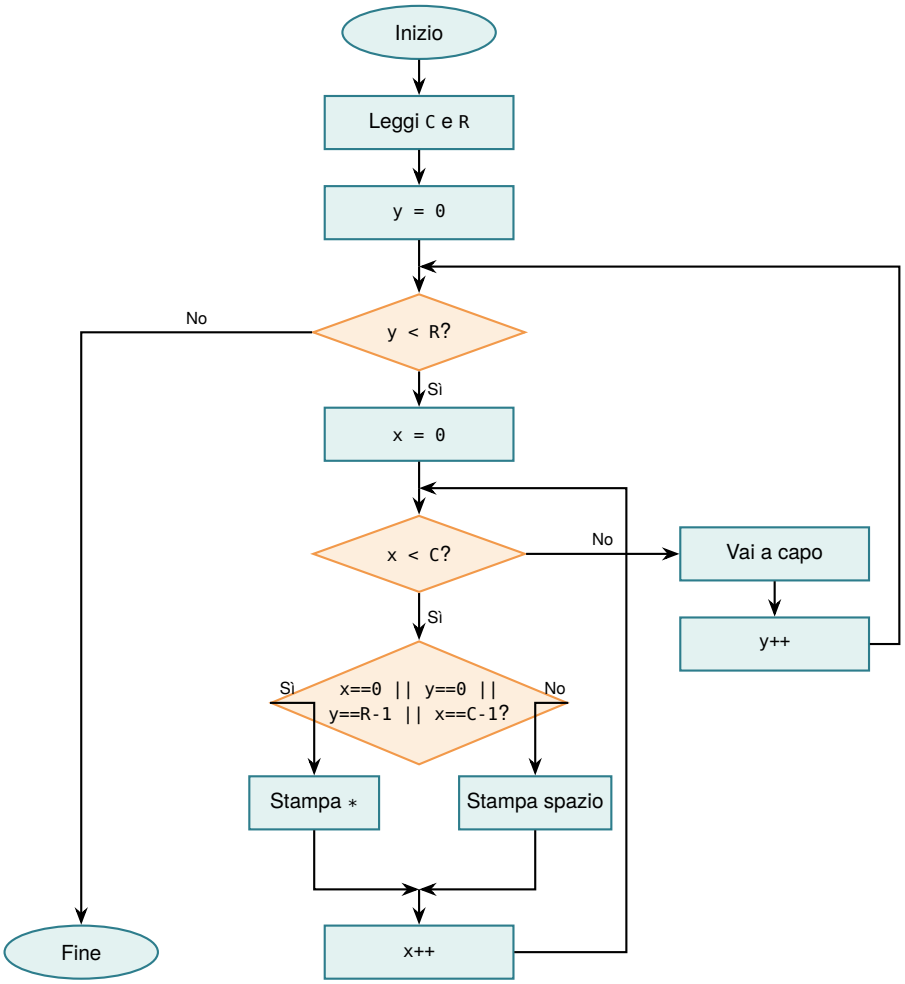


Figura 3: Esercizio 3: dentro il ciclo interno, un `if ... else` sceglie tra asterisco (bordo) e spazio (interno).

4.4 L'indentazione serve a trovare gli errori

In questo esercizio le graffe sono tante: due `for`, un `if`, un `else`. Se il codice è scritto **tutto dritto**, senza indentazione, non si capisce niente, e trovare un errore (una graffa mancante, un punto e virgola di troppo) diventa difficilissimo, **anche per il docente**. Le regole:

- ogni volta che si **apre una graffa**, il codice dentro si sposta a destra di **una tabulazione**;
- la **graffa** va **sotto** la parola che la apre: sotto la `f` del `for`, sotto la `i` dell'`if`;
- il `printf` dentro un `if` che sta dentro due `for` è spostato di **tre tabulazioni** (più quella del `main`).

L'editor di onlinegdb disegna delle **linee verticali** che mostrano dove **inizia e finisce** un blocco: con il codice indentato, si vede subito se manca una graffa. Errori tipici trovati in questo modo: una graffa che apre un `for` mancante; il `printf("\n");` finito **fuori posto** (va dopo il ciclo interno, non dentro); il `return 0;` finito **dopo** l'ultima graffa invece che prima; un punto e virgola dopo la parentesi del `for` o dell'`if`, che svuota il ciclo; una `x` scritta al posto di una `y`.

Nota

Questi disegni con i caratteri sono la base della **grafica**: il prossimo anno, invece degli asterischi, accenderemo dei **pixel**, e spostandoli si possono fare animazioni e semplici videogiochi come **Pong** (le due racchette e la pallina). Un gioco come Pac-Man è più complesso, soprattutto per la grafica.

5 Esercizio 4: il triangolo rettangolo

Esempio

Testo. Scrivere un programma che chieda all'utente di inserire un numero `N` **maggiore di 0** e **minore di 10**. Il programma stampa un **triangolo rettangolo** di asterischi di lato `N`.
Esempio: se `N` vale 5, il programma stampa

```
*
**
***
****
*****
```

Un **triangolo rettangolo** è un triangolo con un **angolo di 90 gradi**. Questo è anche **isoscele**: ha due lati uguali, lunghi `N`.

Attenzione

Rispetta la richiesta. Il programma deve fare **quello che chiede il testo**: chiedere all'utente un numero e stampare il triangolo di quel lato. Un programma che stampa un triangolo fisso con tanti `printf`, magari copiato da un esempio trovato su internet, **non è un altro modo** di risolverlo: è **un'altra cosa**. Se in pizzeria ordini una margherita e ti portano un trancio di salmone, e te lo fanno pagare lo stesso, ti sta bene? Prendere spunto da un esempio va bene, ma va **adattato** a quello che è richiesto.

5.1 Primo problema: il numero tra 1 e 9

"Maggiore di 0 e minore di 10" vuol dire che i numeri ammessi sono **da 1 a 9**: lo 0 non è concesso, e nemmeno il 10.

La condizione del `do while` dice **quando ripetere**, non quando il dato va bene. Il ragionamento si fa in due passi:

1. scriviamo quando il numero è **giusto**: `N > 0 && N < 10`. È "maggiore di 0 **e** minore di 10": la "e" è l'operatore **AND**, `&&`;
2. noi però dobbiamo ripetere quando il numero **non** è giusto: mettiamo tutta la condizione tra parentesi e davanti il **punto esclamativo**, il **NOT**, che **nega**: se una cosa è vera la fa diventare falsa, e se è falsa la fa diventare vera.

```
do
{
    printf("Inserisci il numero: ");
    scanf("%d", &N);
} while ( !((N > 0) && (N < 10)) );
// ripetiamo se il numero NON e' maggiore di 0 e minore di 10
```

Attenzione

Errore tipico: la OR al posto della AND. Scrivere `N > 0 || N < 10` ("maggiore di 0 **oppure** minore di 10") è sbagliato: **ogni** numero è maggiore di 0 oppure minore di 10 (anche -5, anche 42), quindi questa condizione è **sempre vera**. Con la OR il programma **continua a chiedere** oppure accetta tutto: non fa quello che il testo chiede. Ricorda: "oppure" è la **OR**, "e" è la **AND**. E come si chiama l'operatore che fa il **contrario**, che rende falso ciò che è vero e vero ciò che è falso? È il NOT. Il punto esclamativo **non** è un "NOR": è il **NOT**.

Proviamo prima di andare avanti:

```
Inserisci il numero: -5
Inserisci il numero: 0
Inserisci il numero: 15
Inserisci il numero: 42
Inserisci il numero: 5
```

Con -5, 0, 15 e 42 la domanda si ripete; con 5 il programma va avanti. Si possono provare anche tutti gli altri numeri da 1 a 9. Il primo problema, l'acquisizione del dato, è risolto.

Nota

La stessa condizione si può scrivere anche senza il NOT, dicendo direttamente quando il numero è sbagliato: `N < 1 || N > 9` (minore di 1 **oppure** maggiore di 9). Le due scritture sono **equivalenti**.

5.2 Una regola: quante dimensioni, quanti for

Definizione

Quando si disegna qualcosa, il **numero delle dimensioni** del disegno è uguale al **numero dei for annidati**:

- un disegno su **una linea** (1 dimensione), come la riga dell'Esercizio 1: **un for**;
- un disegno sul **piano** (2 dimensioni): un quadrato, un triangolo, un cerchio, una casetta con l'alberello: **due for**, uno annidato nell'altro;
- un oggetto del **mondo reale** (3 dimensioni): **tre for**, un `for` dentro un `for` dentro un altro `for`.

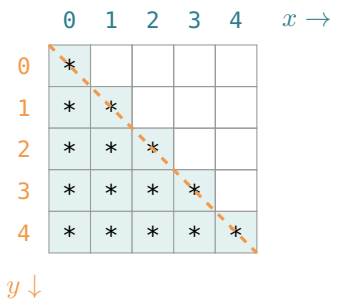
Non è un trucco: è una **verità** che vale per tutti i disegni.

Il triangolo è un disegno in 2 dimensioni: servono **due for annidati**, con la stessa struttura del rettangolo, compreso il `printf("\n");` al termine del **for più interno**.

5.3 Dal quadrato al triangolo

Il **primo passo** è il quadrato pieno: due `for` che vanno tutti e due fino a `N`. Con 5 il programma stampa un quadrato 5 per 5 di asterischi. Non è ancora il triangolo, ma ci siamo vicini.

Ora guardiamo le coordinate, come nel rettangolo vuoto: le colonne `x` da 0 a 4, le righe `y` da 0 a 4.



Quando si stampa l'asterisco? Quando siamo **sotto la diagonale** (o sopra di essa). Prendiamo la riga `y = 2`: gli asterischi sono 3, nelle colonne 0, 1 e 2. Il punto sulla diagonale ha `x = 2` e `y = 2`. Siccome nel ciclo interno andiamo in orizzontale, nella riga `y` si stampano asterischi finché `x` è **minore o uguale** a `y`. Quindi il ciclo interno **non arriva fino a `N`**, ma si ferma a `x <= y`: cambia solo la condizione del `for` interno.

```c
#include <stdio.h>

int main()
{
    int N;

    // ripeto se il numero NON e' maggiore di 0 e minore di 10
    do
    {
        printf("Inserisci il numero: ");
        scanf("%d", &N);
    } while (!(N > 0 && N < 10));

    // un disegno in 2 dimensioni: due for annidati
    for (int y = 0; y < N; y++)
    {
        // nella riga y stampo gli asterischi finche' x <= y
        for (int x = 0; x <= y; x++)
        {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

```
Inserisci il numero: 5
*
**
***
****
*****
```

Nota

È il programma scritto in aula negli ultimi minuti della lezione, provato con 5. Prova a ricostruirlo da solo, partendo dal quadrato pieno, prima di guardarlo.

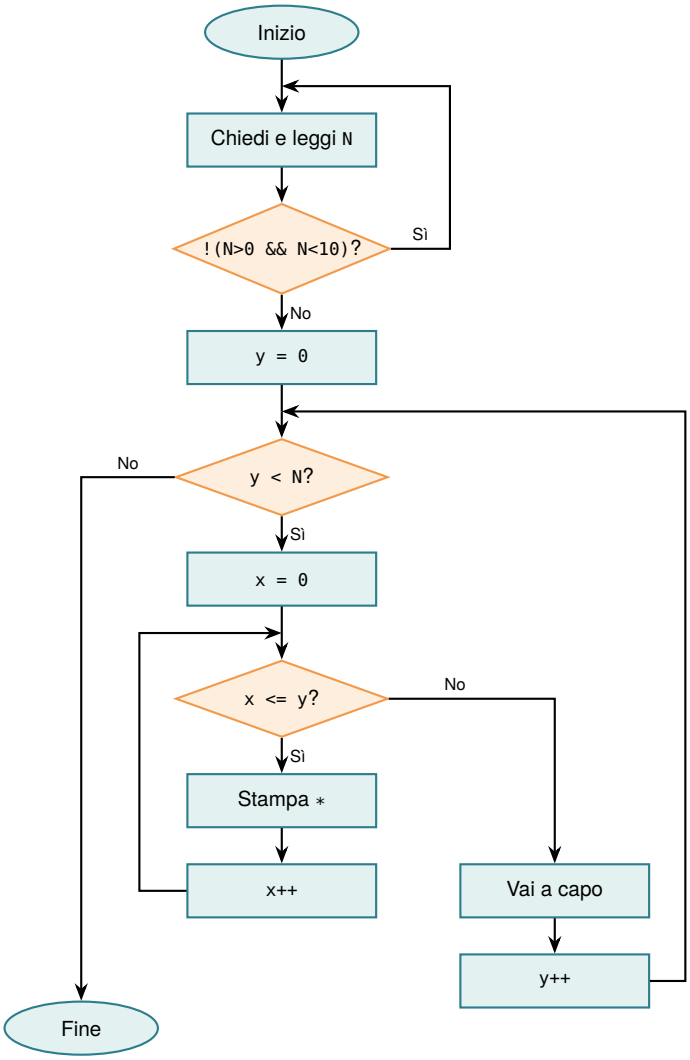


Figura 4: Esercizio 4: il controllo del dato con il NOT, poi i due cicli annidati. Rispetto al rettangolo cambia solo la condizione del ciclo interno: `x <= y`.

6 Cosa ricordare di questa lezione

- Per **chiedere un dato** e ripetere la domanda se è sbagliato: **do while**, con nella condizione **quando ripetere** (il dato sbagliato). "Maggiore di 0" $\Rightarrow$ si ripete con `N < 1`.
- Per ripetere quando un dato **non** rispetta una condizione composta: si scrive la condizione giusta con **AND** (`&&`) e la si nega con il **NOT** (`!`). "E" è AND, "oppure" è OR.
- Ogni tipo di dato ha un **limite**: un `int` di 32 bit va da circa -2 a $+2$ miliardi; oltre c'è l'**overflow**.
- Un problema difficile si **divide in parti**: prima una riga, poi la riga ripetuta.
- Un `for` dentro un `for` è un **for annidato**: l'**esterno** conta le righe (`y`), l'**interno** le colonne (`x`), con **due variabili diverse**. L'**a capo** va dopo il ciclo interno.
- **Dimensioni** del disegno = numero di **for annidati**.
- Per scegliere **che cosa stampare** in ogni punto si usano le **coordinate** `x` e `y`, che partono da 0: l'ultima riga è `R - 1`, l'ultima colonna `C - 1`. Quando non si stampa il carattere, serve un **else** con lo **spazio**.
- Il codice **indentato** permette di trovare subito gli errori; si prova il programma **a ogni passo**.
- Il programma deve fare **quello che chiede il testo**.

Nota

Compito per casa. Su Classroom trovi il **Compito C01**: altre figure con i cicli annidati (triangolo capovolto, triangolo di numeri, triangolo a destra, la X), un conto alla rovescia con il `while` e un menu con lo `switch`. Si continua da dove ci siamo fermati in aula.